

Microcontroladores

Roteiro 7 — Interrupções

Raoni F. S. Teixeira · DENE/UFMT · 1 sessão · 1,0 ponto · grupos de 3

Objetivos desta sessão

Ao final desta sessão o grupo deve ser capaz de:

1. Configurar o Timer0 para gerar uma base de tempo periódica;
2. Escrever uma rotina de tratamento de interrupção correta;
3. Explicar a necessidade de `volatile` em variáveis compartilhadas;
4. Reestruturar o laço principal para não bloquear;
5. Tratar o repique de contato usando a base de tempo periódica.

1 O problema

Seu termostato hoje faz assim:

```
for (;;) {  
    ler_sensor();  
    controlar();  
    atualizar_lcd();  
    __delay_ms(500);  
}
```

Durante os 500 ms de espera o processador não faz nada. Se um botão for pressionado nesse intervalo, o programa não percebe.

🔪 Previsão — registre ANTES de gravar

7.1 — Da sessão anterior: como você faria para ler o sensor a cada 200 ms, atualizar o LCD a cada 300 ms e verificar um botão a cada 10 ms, usando apenas `__delay_ms()`?

Registre sua proposta antes de ver a solução do roteiro.

Conceito central

Com `__delay_ms()`, cada tarefa nova exige recalcular todos os atrasos em função das demais. Com quatro ou cinco tarefas de períodos diferentes, o cálculo vira um quebra-cabeça que nenhuma alteração posterior sobrevive.

A interrupção resolve isso invertendo a relação: em vez de o programa **esperar** o tempo passar, o hardware **avisa** quando passou.

2 Configuração de bancada

Configuração de bancada

A mesma do Roteiro 6. Nenhuma chave precisa ser alterada.

3 Parte 1 — A base de tempo

O Timer0 conta pulsos derivados do clock. Quando estoura, dispara uma interrupção.

$$T_{\text{estouro}} = (2^{16} - \text{precarga}) \times 4 \times T_{\text{osc}} \times \text{prescaler}$$

Com $F_{\text{osc}} = 16 \text{ MHz}$, o timer conta a $F_{\text{osc}}/4 = 4 \text{ MHz}$. Com prescaler 1:64, a contagem avança a 62 500 Hz. Para um tique de 10 ms:

$$\text{contagens} = 0,010 \times 62500 = 625$$

Logo a precarga é $65536 - 625 = 64911$.

■ Divergência do manual

O clock da CPU é 16 MHz. Os config bits do bootloader entregam 48 MHz ao periférico USB e 16 MHz à CPU; os #pragma config da aplicação não têm efeito.

Todo cálculo de tempo desta disciplina parte desse valor — e ele foi obtido por medição em osciloscópio, não por leitura do manual.

1.1 Refaça a conta para um tique de 1 ms. Qual seria a precarga?

Tarefa

1.2 — Configure o Timer0 e escreva a rotina de interrupção:

```
#define TMR0_PRECARGA (65536u - 625u)

static volatile uint8_t g_tique = 0;

static void __interrupt() isr(void)
{
    if (INTCONbits.TMR0IF) {
        INTCONbits.TMR0IF = 0;           /* limpa a flag */
        TMR0H = (uint8_t)(TMR0_PRECARGA >> 8);
        TMR0L = (uint8_t)(TMR0_PRECARGA & 0xFFu);
        g_tique = 1;
    }
}
```

```
}  
}
```

⚠ Atenção

A ordem importa: **limpar a flag** antes de recarregar o timer. Se a flag não for limpa, a interrupção dispara indefinidamente e o programa principal nunca executa.

Escrever TMR0H antes de TMR0L também é obrigatório: o hardware transfere os dois bytes juntos apenas na escrita de TMR0L.

4 Parte 2 — Verificação no LED

Tarefa

2.1 — Faça a interrupção alternar o LED 0 a cada 50 tiques.

Com tique de 10 ms, o LED deve piscar a 1 Hz.

2.2 Meça com cronômetro: dez piscadas devem levar 10 segundos. Quanto levaram?

2.3 Se houver desvio, ele é compatível com a precarga calculada? Que outra fonte de erro pode existir?

5 Parte 3 — volatile

🔪 Previsão — registre ANTES de gravar

3.1 — A variável g_tique é escrita na interrupção e lida no laço principal. O que aconteceria se ela fosse declarada **sem** volatile?

Preveja antes de testar.

Tarefa

3.2 — Remova o volatile, compile com otimização habilitada e observe.

3.3 O programa continuou funcionando? Descreva o que aconteceu.

Conceito central

O compilador analisa o laço principal e conclui, corretamente do ponto de vista dele, que nada ali modifica `g_tique`. A otimização então substitui a leitura repetida por uma leitura única — e o laço trava para sempre.

O compilador não sabe que existe uma interrupção. `volatile` é a forma de dizer: **esta variável muda por fora; releia sempre**.

Este é um dos poucos erros que somem quando se desliga a otimização — e voltam na versão final.

5.1 Acesso não atômico

Observação

Uma variável de 16 bits é lida em duas instruções de 8 bits no PIC18. Se a interrupção ocorrer entre as duas, o valor lido mistura metades de momentos diferentes.

Isso não afeta `g_tique`, que tem 8 bits. Afeta contadores maiores — e a solução é desabilitar a interrupção durante a leitura, ou copiar o valor com GIE desligado.

3.4 Dê um exemplo, no contexto do termostato, de variável em que esse problema apareceria.

6 Parte 4 — Reestruturação do laço

Tarefa

4.1 — Reescreva o laço principal sem nenhum `__delay_ms()`:

```
for (;;) {
    if (!g_tique) continue;
    g_tique = 0;

    /* --- a cada 10 ms --- */
    verificar_botoes();

    contador_sensor += 10;
    contador_lcd    += 10;

    if (contador_sensor >= 200u) {
        contador_sensor = 0;
        ler_sensor();
        controlar();
    }

    if (contador_lcd >= 300u) {
        contador_lcd = 0;
        atualizar_lcd();
    }
}
```

```
}
}
```

4.2 Compare com a sua proposta do item 7.1. O que muda quando é preciso acrescentar uma quarta tarefa, com período de 1 segundo?

4.3 O botão agora responde imediatamente? Teste apertando durante a atualização do LCD.

Conceito central

Esta estrutura — interrupção gera tique, laço principal distribui trabalho — é o esqueleto de praticamente todo sistema embarcado sem sistema operacional.

Em ARM Cortex-M o mecanismo se chama SysTick e a configuração é diferente, mas o raciocínio é idêntico. É o conteúdo desta disciplina que mais transfere de plataforma.

7 Parte 5 — Debounce: a dívida do Roteiro 2

No Roteiro 2 vocês encontraram o **repique de contato**: um botão apertado dez vezes produzia mais de dez contagens. Naquele momento a solução foi adiada — faltava a ferramenta. Agora ela existe.

7.1 Por que o repique acontece

A lâmina metálica do botão não fecha o contato de uma vez. Ela bate, quica e oscila por alguns milissegundos antes de estabilizar, produzindo uma rajada de transições. Um laço que lê o pino milhares de vezes por segundo enxerga cada quique como um acionamento distinto.

Dispositivo	Duração típica do repique
Botão de painel	1 a 10 ms
Chave DIP	1 a 5 ms
Relé eletromecânico	5 a 20 ms
Contator de potência	10 a 50 ms

Tabela 1: Ordem de grandeza do repique em diferentes contatos mecânicos.

Conceito central

O repique não é defeito do componente nem ruído elétrico: é o comportamento mecânico normal

de qualquer contato que se fecha por impacto. Ele existe do botão de campainha ao contator de uma subestação.

O que muda de um caso para outro é a **escala de tempo** — e é por isso que o tratamento precisa ser temporizado, não apenas lógico.

7.2 A solução por confirmação temporizada

A ideia: só aceitar uma leitura depois que ela se repetir por várias amostragens consecutivas.

Com o tique de 10 ms já disponível, três leituras iguais equivalem a 30 ms de estabilidade — acima do repique de um botão comum, conforme a Tabela 1.

🔪 Previsão — registre ANTES de gravar

5.1 — Antes de implementar:

(a) Se o tique fosse de 1 ms em vez de 10 ms, quantas confirmações seriam necessárias para a mesma proteção?

(b) O que aconteceria se o número de confirmações fosse alto demais — digamos, 50 tiques?

Tarefa

5.2 — Implemente o tratamento como uma função chamada **a cada tique**:

```
#define CONFIRMACOES 3

static uint8_t g_estado_estavel = 1; /* 1 = solto */
static uint8_t g_candidato      = 1;
static uint8_t g_contador       = 0;
static uint8_t g_evento         = 0;

void botao_mostrar(void)
{
    uint8_t leitura = PORTBbits.RB0; /* 0 = pressionado */

    if (leitura != g_candidato) {
        g_candidato = leitura;
        g_contador = 0;
        return;
    }

    if (g_contador < CONFIRMACOES) {
        g_contador++;
        if (g_contador == CONFIRMACOES) {
            /* estado confirmado: registra apenas a BORDA */
            if (leitura == 0u && g_estado_estavel == 1u) {
```

```

        g_evento = 1;
    }
    g_estado_estavel = leitura;
}
}
}

```

No laço principal, consuma o evento uma única vez:

```

if (g_evento) {
    g_evento = 0;
    contador_apertos++;
}

```

5.3 Por que a função registra a **borda** (transição de solto para pressionado) em vez do estado? O que aconteceria se contasse o estado?

Tarefa

5.4 — Repita o experimento do Roteiro 2: aperte o botão dez vezes, devagar, e confira a contagem. Faça três tentativas.

Tentativa	Sem debounce (R2)	Com debounce
1		
2		
3		

5.5 A contagem ficou exata? Se ainda houver erro, ele é para mais ou para menos? O que isso indica?

Tarefa

5.6 — Reduza CONFIRMACOES para 1 e repita. Depois aumente para 20 e repita.

CONFIRMACOES = 1	Contagem obtida:	
CONFIRMACOES = 20	Contagem obtida:	

5.7 Com 20 confirmações (200 ms), o que acontece se você apertar o botão rapidamente? Relacione com a previsão 5.1(b).

Conceito central

O debounce é um compromisso: tempo curto demais não filtra o repique; tempo longo demais descarta acionamentos legítimos. O valor correto depende do contato usado e da velocidade esperada de operação.

Repare que esta solução **não bloqueia**. A abordagem ingênua — `if (botao) { __delay_ms(50); ... }` — funciona, mas paralisa o sistema a cada toque. Em um termostato isso significa parar de controlar a temperatura enquanto o operador mexe no painel.

Observação

Existe também debounce por hardware, com filtro RC ou *Schmitt trigger*. É mais caro em componentes e menos flexível, mas não consome tempo de processamento — a escolha depende de quantos contatos o sistema tem e de quão ocupado está o processador.

8 Teste de aceitação

- ☐ LED piscando a 1 Hz por interrupção, confirmado com cronômetro.
- ☐ Laço principal sem nenhum `__delay_ms()`.
- ☐ Botão respondendo durante a atualização do LCD.
- ☐ Contagem de dez apertos exata em três tentativas, com debounce.

9 Entrega e critério

Peso	Item
0,2	Teste de aceitação aprovado
0,1	Proposta 7.1 registrada antes da solução, e comparação 4.2
0,1	Cálculo 1.1 e medição 2.2
0,3	Previsão 3.1 e resultado 3.3 sobre <code>volatile</code>
0,3	Previsão 5.1, tabelas 5.4 e 5.6, e respostas 5.3 e 5.7

Tabela 2: Distribuição do ponto do Roteiro 7.

10 Armadilhas frequentes

Sintoma	Causa provável
Programa trava logo após iniciar	Flag TMR0IF não foi limpa na ISR
Laço nunca executa	GIE desligado, ou volatile ausente com otimização
Período errado por fator 2 ou 4	Precaler diferente do usado no cálculo
Tempo instável	Precarga escrita em TMR0L antes de TMR0H
LCD com caracteres perdidos	Interrupção ocorrendo no meio da escrita
Contagem ainda errada com debounce	Evento não zerado após consumo
Botão “não responde”	CONFIRMACOES alto demais para a velocidade do toque
Contagem dobrada	Registrou estado em vez de borda

Tabela 3: Diagnóstico rápido do Roteiro 7.

11 Para a próxima sessão

Tarefa

Até agora, para saber o que o sistema está fazendo, é preciso olhar o LCD — que mostra apenas o instante presente.

Traga escrito: como você faria para registrar a temperatura a cada segundo durante dez minutos, e depois analisar a curva? O que seria necessário?